ASSIGNMENT 1 – ADVANCED GEN AI

Using a Foundation Model for Multiple Tasks

Python hands-on — Visualize attention with BertViz

Running an LLM with Hugging Face Locally

Using IBM Granite via Hugging Face

Using Inference API (No Download Required)

GAN

VAE

Stabel Diffusion

Prompting 5 Programs – Zero Shot, One Shot , Few Shot , COT, TOT

General Instructions

- Run all programs in Google Colab.
- Install libraries using !pip install before importing.
- Follow the steps given for each question.
- Print your output clearly so it is easy to read.

QUESTION 1 — Check if a Sentence is Positive or Negative

What You Have to Do

Write a Python program that takes 4 simple sentences and checks whether each sentence is POSITIVE or NEGATIVE. Print the result along with how confident the model is (the score).

The 4 Sentences to Use

```
sentences = [
    "I love this product, it is amazing!",
    "This is the worst experience I have ever had.",
    "The food was really good and tasty.",
    "I am very disappointed with the service."
]
```

Steps to Follow

Step 1: Install the transformers library. Write: !pip install transformers -q

Step 2: Import pipeline from transformers.

Step 3: Load the model using pipeline("sentiment-analysis").

Step 4: Loop through each sentence and pass it to the model.

Step 5: Print: sentence number, the label (POSITIVE or NEGATIVE), and the score.

What Your Output Should Look Like

```
Sentence 1: I love this product, it is amazing!
  Result : POSITIVE
  Score  : 0.9998

Sentence 2: This is the worst experience I have ever had.
  Result : NEGATIVE
  Score  : 0.9987
```

Note: *The score is a number between 0 and 1. Closer to 1 means more confident.*

QUESTION 2 — Complete a Sentence using GPT-2

What You Have to Do

Write a Python program that takes a short starting sentence and uses GPT-2 to continue and complete it. Generate 2 different completions for each starting sentence.

The 3 Starting Sentences to Use

```
prompts = [
    "Artificial intelligence is",
    "The best way to learn programming is",
    "In the future, robots will"
]
```

Steps to Follow

Step 1: Install transformers. Write: !pip install transformers -q

Step 2: Import pipeline from transformers.

Step 3: Load GPT-2 using pipeline("text-generation", model="gpt2").

Step 4: For each prompt, call the generator with max_new_tokens=40 and num_return_sequences=2.

Step 5: Print each prompt and its 2 generated completions clearly.

What Your Output Should Look Like

```
Prompt: Artificial intelligence is
  Completion 1: Artificial intelligence is changing the way we
                work and live every single day...
  Completion 2: Artificial intelligence is a technology that
                allows machines to learn from data...
```

Note: *Set do_sample=True and temperature=0.8 so each completion is different.*

QUESTION 3 — Predict the Missing Word in a Sentence

What You Have to Do

Write a Python program that uses BERT to predict the missing word in a sentence. The missing word is shown as [MASK]. Print the top 3 predicted words with their confidence percentages.

The 4 Sentences to Use

```
sentences = [
    "The student is studying for the [MASK] exam.",
    "She cooked a delicious [MASK] for dinner.",
    "He went to the [MASK] to buy vegetables.",
    "The doctor gave the patient a [MASK]."
]
```

Steps to Follow

Step 1: Install transformers. Write: !pip install transformers -q

Step 2: Import pipeline from transformers.

Step 3: Load the model using pipeline("fill-mask", model="bert-base-uncased").

Step 4: For each sentence, pass it to the model with top_k=3 to get the top 3 predictions.

Step 5: For each prediction, print the word and the score multiplied by 100 to show percentage.

What Your Output Should Look Like

```
Sentence: The student is studying for the [MASK] exam.
  Prediction 1: final       — 45.23%
  Prediction 2: board       — 22.11%
  Prediction 3: entrance    — 12.34%
```

Note: *[MASK] must be written in CAPITAL letters exactly as shown. BERT recognises this special token.*

QUESTION 4 — Count How Many Tokens are in a Sentence

What You Have to Do

Write a Python program that counts how many tokens a sentence has when processed by GPT-4. Also decode the tokens back to words and print them. Use the tiktoken library for this.

The 3 Sentences to Use

```
sentences = [
```

```
    "Hello, how are you?",
    "Machine learning is a part of artificial intelligence.",
    "India is a beautiful country with many languages and cultures."
]
```

Steps to Follow

Step 1: Install tiktoken. Write: !pip install tiktoken -q

Step 2: Import tiktoken.

Step 3: Load GPT-4 encoding using tiktoken.encoding_for_model("gpt-4").

Step 4: For each sentence, use enc.encode(sentence) to get the token IDs.

Step 5: Print the sentence, the number of tokens (len of the list), and the token ID list.

Step 6: Use enc.decode(tokens) to convert back and print the decoded text.

What Your Output Should Look Like

```
Sentence : Hello, how are you?
Tokens   : [15496, 11, 703, 389, 345, 30]
Count    : 6 tokens
Decoded  : Hello, how are you?
```

Note: *Tokens are just numbers. The model never sees your English words — only these numbers.*

QUESTION 5 — Search for Top AI Models on Hugging Face

What You Have to Do

Write a Python program that searches Hugging Face for the top 3 most downloaded models for two tasks: text-generation and sentiment-analysis. Print the model name and download count for each.

The 2 Tasks to Search

```
tasks = [
    "text-generation",
    "sentiment-analysis"
]
```

Steps to Follow

Step 1: Install huggingface_hub. Write: !pip install huggingface_hub -q

Step 2: Import HfApi from huggingface_hub.

Step 3: Create the API object: api = HfApi()

Step 4: For each task, call api.list_models() with pipeline_tag=task, sort="downloads", limit=3.

Step 5: Wrap the result in list() to convert it from a generator to a normal list.

Step 6: Loop through the results. For each model print: rank number, modelId, and download count. Use getattr(m, "downloads", 0) or 0 to safely get the download count.

What Your Output Should Look Like

```
Task: text-generation
  Rank 1 : gpt2                    —  38,234,567 downloads
  Rank 2 : meta-llama/Llama-2-7b   —  29,456,789 downloads
  Rank 3 : mistralai/Mistral-7B     —  18,345,678 downloads

Task: sentiment-analysis
  Rank 1 : distilbert-base-uncased  —  25,123,456 downloads
  Rank 2 : cardiffnlp/twitter-roberta— 12,456,789 downloads
  Rank 3 : nlptown/bert-multilingual —  8,234,567 downloads
```

Note: *Use {downloads:,} formatting to add commas to large numbers for readability.*